Build Steps

Part 1 of 5 · Set up your agent in the new builder

1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

Agent name Patient Intake Summariser Copilot
Description Converts free-text intake notes (synthetic, clearly fictional) into a structured pre-consult summary for clinicians, with red flags surfaced first.
Solution / Environment Healthcare Solution · Default environment
Icon Use a recognisable colour and emoji, or upload a brand icon

2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

Reasoning is built in

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

Choosing a model

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

In healthcare, model choice is a governance decision: confirm data residency for your tenant, keep the agent on synthetic data for the build, and apply Microsoft Purview labels so no real patient data ever reaches the model.

3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

```
You are the Patient Intake Summariser Copilot. All data is synthetic and clearly fictional.
You help clinicians prepare for consults.

Rules:
- Use the uploaded intake dataset as your single source of truth.
- Never expose a patient name; refer to the pseudonymised patient reference only.
- Return a structured pre-consult summary: presenting complaint, symptoms, history, red flags.
- Always surface red flags at the top of the summary.
- Do not give a diagnosis or treatment advice; summarise what the intake note records.
- For follow-up, call the 'Create Clinician Task in Planner' tool rather than pinging in plain text.
```

Build Steps

Part 2 of 5 · Ground it in your data

4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **Healthcare_Intake_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Pseudonymised reference (patient reference, intake date, clinic)
- Presenting data (free-text intake note, presenting complaint)
- Structured signals (symptom tags, red flag flag, triage band)
- Routing (assigned clinician, status)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ links the pseudonymised reference to the clinician's Planner and Teams without exposing identity, so a task lands with the right clinician while patient detail stays inside the governed boundary. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

Build Steps

Part 3 of 5 · Give it skills and tools

6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

Skill name: Summarise Patient Intake
Description: Reads a free-text intake note for a pseudonymised patient and returns a structured pre-consult summary with red flags highlighted at the top.
When to use it (intent hints the agent reasons over):

- Summarise the intake for PT-007
- Give me a pre-consult brief
- Brief me on my next patient
- What's the summary for PT-022

Structured steps:

- Identify the patient reference (never a name)
- Read the free-text intake note from knowledge
- Extract presenting complaint, symptoms, relevant history
- Surface any red flag at the very top of the summary
- Return the structured summary and offer to create a clinician task
- Call 'Create Clinician Task in Planner' if asked

7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action.** A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build uses a Power Automate flow wrapping the Planner connector because it does the work in one call: create the task, notify the clinician, and return a task URL.

Tool name: Create Clinician Task in Planner
Type: Power Automate flow over the Microsoft Planner connector (Create a task), plus a Teams adaptive card to the clinician
Description (the model reads this to decide when to call): Creates a Planner task for clinician review with the pseudonymised patient reference and red flag context, and pings the clinician in Teams.
Inputs the agent collects or infers:

- patientReference — text, pseudonymised, never a name
- redFlagContext — text, the red flag in the intake note's own words
- triageBand — text, one of Routine / Soon / Urgent
- assignedClinician — text, from the routing column

Outputs returned to the agent:

- taskUrl — text, e.g. https://tasks.office.com/.../task/AbC123XyZ
- status — text, e.g. Created
- clinicianNotified — yes/no

7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (Healthcare Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

Trigger — *When an agent calls the flow*. Add four text input parameters, named exactly: `patientReference`, `redFlagContext`, `triageBand`, `assignedClinician`.

– **Action 1 – create the Planner task.** Microsoft Planner *Create a task* in the **Clinician Review** plan, the assigned clinician's bucket. Map: Title = `concat('Review ', patientReference, ' (', triageBand, ')')`, Assigned to = assignedClinician. Capture the returned task id.
– **Action 2 – add the detail.** Planner *Update task details*, Description = redFlagContext (red flag first). Build `taskUrl = concat('https://tasks.office.com/.../task/', body('Create_a_task')?['id'])`.
– **Action 3 – notify the clinician.** Microsoft Teams *Post card in a chat or channel* to the assigned clinician, using an adaptive card that shows the patient reference, triage band, red flag context and the task link.
– **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs taskUrl = `variables('taskUrl')`, status = 'Created', clinicianNotified = true.
– **Save** the flow as `Healthcare – Create Clinician Task`.

7.2 · Register and map the tool in your agent

– Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
– Set the display name to **Create Clinician Task in Planner** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
– For each input set **Fill using**: patientReference → *dynamically* from the reference the skill is summarising; assignedClinician → *dynamically* from the routing column; redFlagContext and triageBand → **Fill with AI**, and add an input description so the model extracts the right value (for triageBand, list Routine/Soon/Urgent and what each means).
– Confirm `taskUrl` is returned to the agent so the skill can read it back — satisfying your instruction rule "call the tool rather than pinging in plain text".
– **Connection & auth:** set the Planner and Teams connections the flow uses. The connectors chosen here are exactly what your DLP policy must permit (see Step 10).

7.3 · Worked example — what flows in and out

When a clinician says *“create a task for PT-022, chest pain red flag, urgent”*, the model fills the inputs and calls the tool with this payload:

```
{
  "patientReference":  "PT-022",
  "redFlagContext":    "Chest pain on exertion, radiating to left arm",
  "triageBand":        "Urgent",
  "assignedClinician": "Dr. Patel"
}
```

The flow creates the task, posts the card, and returns:

```
{
  "taskUrl":           "https://tasks.office.com/.../task/AbC123XyZ",
  "status":            "Created",
  "clinicianNotified": true
}
```

The agent then replies in natural language, quoting the real task: *“I’ve created an Urgent Planner task for PT-022 (chest pain red flag) and notified Dr. Patel.”*

7.4 · Test the tool before you trust it

– **In isolation:** open the flow in Power Automate, choose *Test* > *Manually*, supply the four inputs, run it, and confirm a task appears in the Clinician Review plan, a card posts to the clinician, and the run returns all three outputs.
– **End to end:** in **Preview** (Step 11) type “create a clinician task for PT-022 with the red flag chest pain context” and check the activity view shows Create Clinician Task in Planner firing with the inputs the model inferred and the task URL it read back.

Build Steps

Part 4 of 5 · Extend, remember and govern

8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect a **Clinical Coding Agent** that proposes SNOMED or ICD codes for the structured summary, keeping coding separate from summarisation while sharing the same pseudonymised reference.

9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- The clinician's review queue only, never patient detail beyond the session
- The clinician's preferred summary format, within retention and Purview policy

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

10 Govern your agent

Open the agent settings and lock these down before real users arrive:

Authentication: use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.
Content moderation: set the moderation level (start at Medium) and review it for your audience.
Data loss prevention: confirm your DLP policy covers the connectors your tools use — for Create Clinician Task in Planner that means the Planner and Teams connectors must sit in the same DLP group, or the tool is blocked at runtime.
Channels and access: review which channels are enabled and who can use the agent.

Build Steps

Part 5 of 5 · Preview, evaluate and publish

11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and what each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows

12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

Test set: Patient Intake Summariser · Baseline v1
Input: Summarise the intake for PT-007
Expect: Returns a structured summary using the pseudonymised reference, red flag first.
Input: Show all intakes flagged red this week
Expect: Returns rows where the red flag is set for the week, no patient names.
Input: Create a clinician task for PT-022 with the red flag chest pain context
Expect: Triggers the summary skill and calls Create Clinician Task in Planner.

13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

Try These Live

Sample queries to test your agent during the build-along.

Pre-consult Briefs
Summarise the intake for PT-007. · Give me a pre-consult brief for my next three patients.

Safety Surveillance
Show all intakes flagged red this week. · Are any red flags recurring (same presenting complaint, same week)?

Governed Action
Create a clinician task for PT-022 with the red flag chest pain context.
Try a paraphrase: 'ping the on-call clinician about PT-011, urgent'.

How to Think About This Agent

This agent replaces	With
• Reading raw intake notes mid-clinic • Re-typing red flags into Planner or paper • Ad-hoc Teams pings with patient context in plain text	✓ Consistent pre-consult summaries on demand ✓ Red flags surfaced first, every time ✓ Planner tasks created with a full audit trail

Trusted, governed, enterprise AI.
Built once, used everywhere.

Tip for real-world use
Use Microsoft Purview sensitivity labels on the knowledge file and the agent so it cannot surface in unmanaged channels. Pair with strict DLP: block the agent from any external surface, and require Entra authentication on every chat.

You've built a Copilot Studio agent ✓
You now have an AI agent that can:

- Render structured pre-consult briefs
- Flag red-flag intakes for review
- File clinician tasks in Planner
- Keep clinicians always in the loop